

Project Executable Files

Date	1 October 2026
Team ID	SWTID-2026-4934
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	3 Marks

Project Executable Files

This section lists everything needed to run and check the project. The solution is built by configuration in ServiceNow: one UI Policy with two UI Policy Actions, and three Client Scripts on the Incident table. There is no separate application to install or run. The evaluator opens the instance, the UI Policy and the Client Scripts, and then tries the behaviour on the Incident form and list using the access details and steps below.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes (the three Client Script listings are in the Appendix of the Project Documentation; the UI Policy is built by configuration in ServiceNow)
2	README / Setup Guide (instructions to run the project)	Yes (README.md is added to the GitHub repository)
3	requirements.txt / package.json / dependency file	NA
4	Database schema / seed files (if applicable)	NA (uses the built-in ServiceNow Incident table)
5	Environment configuration file (.env.example or similar)	NA
6	Deployed application URL (if hosted)	Yes (ServiceNow developer instance, see Step 3)
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes (Section 6 of the Project Documentation, test cases TC-01 to TC-05)
10	Demo video or walkthrough (if required)	Yes (link in Step 3)

Step 2: File / Folder Structure

Overview of the project's file and folder structure:

```

/GitHub repository root (https://github.com/Rash6906/Implement-Client-Script-and-UI-Policy-Incident/tree/main)
/1. Brainstorming & Ideation - Problem Statement, Empathy Map Canvas, Brainstorming
/2. Requirement Analysis - Data Flow Diagram, Solution Requirements, Technology Stack, Customer Journey Map
/3. Project Design Phase - Proposed Solution, Solution Architecture, Problem-Solution Fit
/4. Project Planning Phase - Project Planning (task breakdown, Tasks 1 to 6)
/5. Project Development Phase - Coding & Solution (UI Policy configuration and the onChange, onSubmit and onCellEdit Client Scripts)
/6. Project Testing - Functional testing (TC-01 to TC-05) and performance note
  
```

/7. Project Documentation - Project Executable Files, Project Documentation
 /8. Project Demonstration - Demo video / walkthrough
 README.md - Repository overview

On the ServiceNow instance (no source files):

UI Policy: High Impact Control (System UI > UI Policies), with actions on Assignment group and Urgency

Client Script: Auto set urgency for high impact (onChange, Impact)

Client Script: Prevent save if assigned to missing (onSubmit)

Client Script: Prevent state change via list edit (onCellEdit, State)

Step 3: Deployment / Access Details

Hosted / Deployed URL	https://dev448044.service-now.com (ServiceNow developer instance)
Login Credentials (Demo)	Username: puttarashmi6906@gmail.com Password: Benq_6158
Platform / Hosting Provider	ServiceNow (developer instance, cloud hosted)
Repository Link	https://github.com/Rash6906/Implement-Client-Script-and-UI-Policy-Incident-.git
Demo Video Link	https://drive.google.com/file/d/1lsgcZhEpz95CWjBMqLH3ClnFkmdDAWJ/view?usp=sharing

Step 4: Run Instructions

Step-by-step instructions for the evaluator to run the project:

Pre-requisites:

- A web browser (Chrome or Edge) and an internet connection
- Access to the ServiceNow instance above with administrative or configuration access

Steps:

1. Log in to the ServiceNow instance.
2. Open All > System UI > UI Policies and open "High Impact Control". Check that the table is Incident, Active is selected, the condition is Impact is 1 - High and Reverse if false is selected. In the UI Policy Actions list, check the Assignment group action (Mandatory: True) and the Urgency action (Read only: True).
3. Open All > System UI > Client Scripts and check that the three scripts are active: "Auto set urgency for high impact", "Prevent save if assigned to missing" and "Prevent state change via list edit".
4. Open Incident > Create New. Fill in Caller and Short description, then set Impact to 1 - High. Urgency should change to 1 - High and become read-only. Assignment group becomes mandatory, so select one.
5. Leave Assigned to empty and click Submit. The save should be blocked, and the message "Assigned To is mandatory for High impact incidents." should appear under Assigned to.
6. Select a user in Assigned to and click Submit. The incident should now be saved.
7. Open an incident that has High impact, change Impact to 2 - Medium, and check that Urgency is editable again and Assigned to is no longer enforced.
8. Open Incident > Open (or All), double-click the State value of any incident in the list, and check that an alert appears and State stays unchanged.
9. Open an incident form, change State (for example to In Progress) and click Update. The change should be saved.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	The controls are client-side. Records created outside the form and list, for example through imports or integrations, are not checked by them.	Planned as a future improvement (server-side controls)
2	The onChange script sets Urgency to High when Impact becomes High but does not change it back when Impact is lowered; the field only becomes editable again.	Planned as a future improvement

S.No	Known Issue / Limitation	Workaround / Status
3	The onCellEdit script blocks every list edit of State for all users, with no role-based exception.	Planned as a future improvement
4	Assigned to is enforced when the user clicks Submit, so it is not shown as mandatory while the form is being filled in.	Planned as a future improvement
5	Load with many users and CPU / memory use have not been measured.	Planned as future testing
6	A developer instance can go to sleep after a period of inactivity.	Wake the instance from the ServiceNow developer portal before demonstrating